

Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence - IT3021

Assignment 1

IT23864238 – Karunanayake R.M.T.D

Contents

1 Declaration	3
2 Dataset Selection	3
3 Preparation of Data Source	4
3.1 Source Tables Description	4
3.2 Entity Relationship Diagram	5
4 High Level Design Architecture	6
4.1 Data Storage Snapshots from SSMS	7
5 Data Warehouse design and Development	8
5.1 Dimensional Model.....	8
5.2 Star Schema Diagram	8
5.3 Fact Table: FactAppointments	8
5.4 Dimension Tables.....	9
5. 6 Hierarchy Support.....	10
5.7 Design Assumptions	10
6 ETL Development	10
6.1 Data Extraction & Staging Area Loading.....	10
6.1.1 Extraction Components Used.....	10
6.1.2 OnPreExecute Event Handler.....	12
6.2 Data Extraction and Data Warehouse Loading.....	12
6.2.1 Slowly Changing Dimension – DimPatient (SCD Type 2).....	13
6.2.2 Fact Table Loading – LoadFactAppointment.....	15
6.2.3 UpdateAccumulatingFact.....	15
7 Conclusion	18

1 Declaration

I declare that this project report or part of it was not a copy of document done by any organization, university and other institute or a previous student project at SLIIT and was not copied from the internet or other resources

2 Dataset Selection

Data Set Name : Medical Center Management System

Provided by : kaggle.com

Source link : [Healthcare Clinic Dataset](#)

This is a dataset sourced from an established Healthcare Clinic management system that frequently handles a high volume of patient appointments, medical treatments, and clinical operations across multiple clinic locations.

Patient management and appointment scheduling are very widely used operational processes to attract new patients and to retain and enforce continuity of care for existing patients.

The dataset has information on patient appointments with doctors across various clinics, the treatments administered, prescriptions issued, and payments processed for each visit. The features of this dataset allow visualization of data in various perspectives such as clinic-based, doctor specialization-based, and patient demographic-based too. The original dataset contains eight interrelated tables with information on Patients, Doctors, Clinics, Appointments made by patients, and the rest are all dependent and transactional tables of the above mentioned four subject areas, namely Payments, Treatments, Prescriptions, and Staff.

The dataset includes approximately 100,000 appointment records, supported by 20,000 patient records and 500 doctor records, providing sufficient data volume for analysis and data warehouse implementation.

Additionally, a separate flat file containing patient insurance details such as Insurance Status, Insurance Provider, and Coverage Level is also used as a secondary data source alongside the main database.

The Medical Center dataset was selected for the following reasons:

- Multi-entity relationships: The dataset has 8 interrelated tables with primary and foreign key relationships, enabling rich dimensional modeling.
- Multiple data sources: The OLTP database is supplemented by an external insurance flat file, demonstrating heterogeneous ETL.
- Analytical richness: Supports analysis of appointment trends, payment revenues, treatment costs, doctor performance, and patient demographics.
- SCD applicability: Patient insurance data changes over time, making it ideal for implementing SCD Type 2.

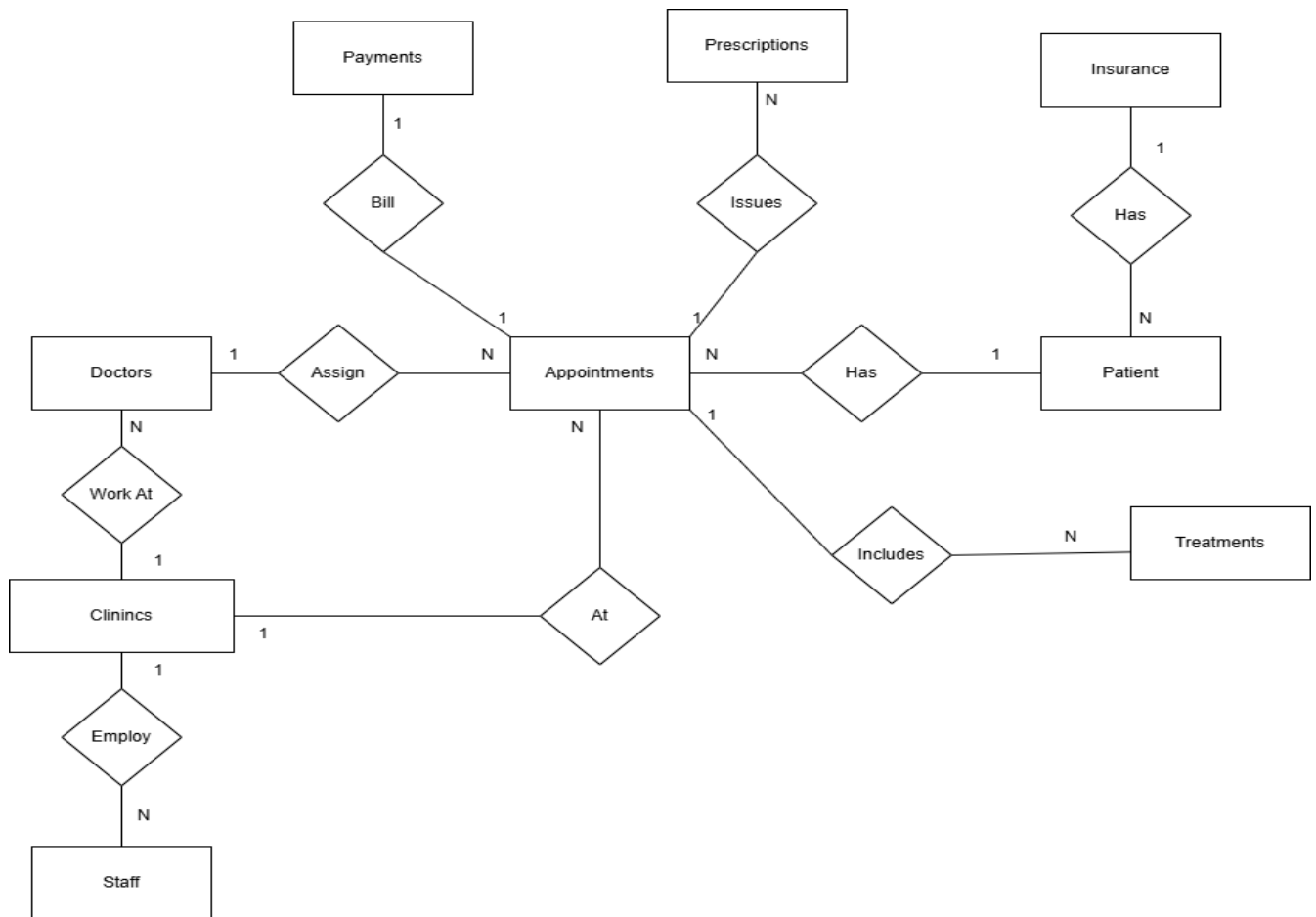
3 Preparation of Data Source

The source data for this project consists of a custom SQL Server OLTP database (MedicalCenter_DB) containing 8 tables, and an external flat file containing patient insurance information (patient_insurance.txt). The OLTP database was designed from scratch to model a realistic medical center scenario.

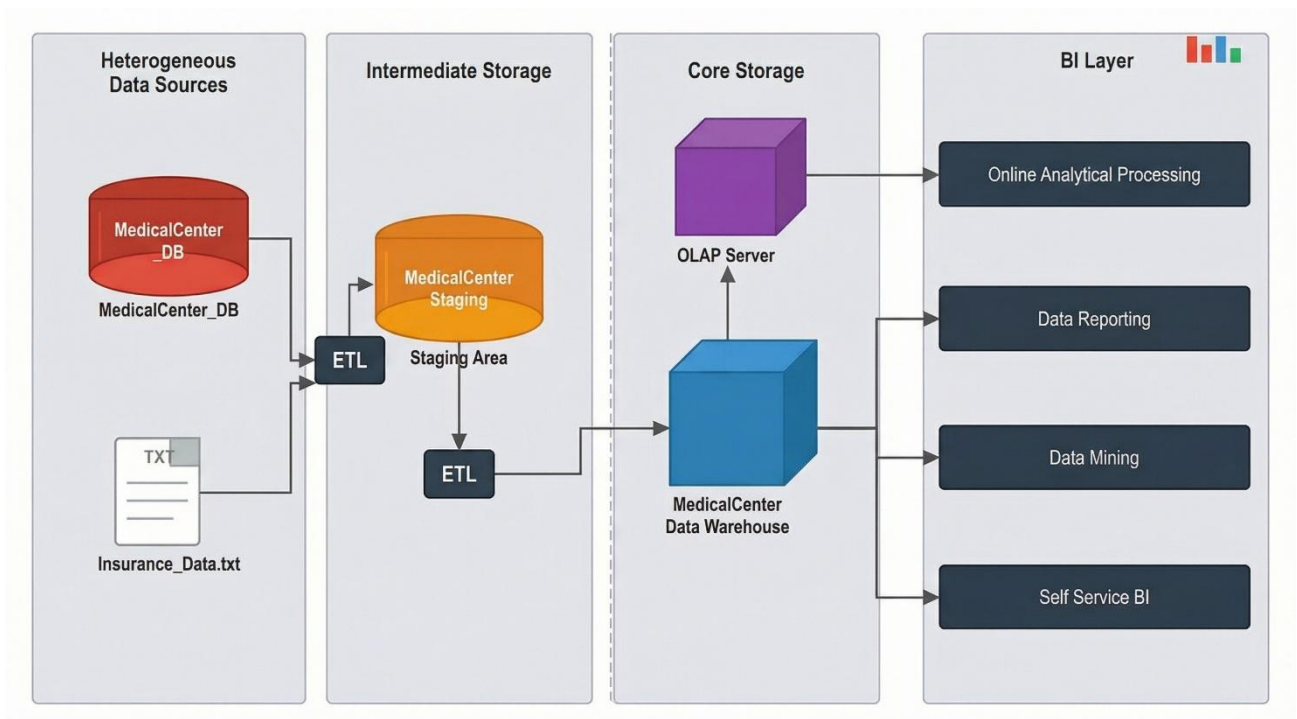
3.1 Source Tables Description

Table Name	Primary Key	Description	Key Attributes
CSV File Tables			
patients	patient_id	Stores patient demographic and contact details	full_name, gender, DOB, blood_group, registration_date,
doctors	doctor_id	Medical professionals and their specializations	full_name, specialty, email, clinic_id (FK)
clinics	clinic_id	Clinic location and operational details	clinic_name, city, state, clinic_type, opening_hours
appointments	appointment_id	Core transactional table linking patients, doctors, clinics	patient_id, doctor_id, clinic_id, appointment_date, status, reason for visit
payments	payment_id	Financial transactions per appointment	payment_amount, payment_method, payment_status
treatments	treatment_id	Medical treatments given per appointment	treatment_type, treatment_cost, treatment_status
prescriptions	prescription_id	Medicine prescriptions per appointment	medicine_name, instructions
staff	staff_id	staff members at each clinic	full_name, role, clinic_id (FK)
Flat File Table			
insurance	patient_id	Patients' insurances details	Insurance_status, insurance_provider, coverage_level

3.2 Entity Relationship Diagram



4 High Level Design Architecture



Layer 1 – Data Sources

The source layer is made up of two non-homogenous sources of data:

- **MedicalCenter_DB (SQL Server OLTP):** This is the main transactional database that will hold all the 8 operational database tables. In SSIS, OLE DB Source connectors are used to extract data.
- **Insurance Flat File (.txt):** This is an external text file that holds the insurance status, provider and the coverage level of the patients. This is an addition to the OLTP database that is loaded using Flat File Source in SSIS.

Layer 2 -Staging Area (MedicalCenter_Staging)

The staging database acts as an intermediate storage layer. All source data are loaded into their respective staging tables (**stg_patients**, **stg_doctors**, **stg_clinics**, **stg_appointments**, **stg_payments**, **stg_prescriptions**, **stg_treatments**, **stg_staff**, **stg_insurance**) and then any form of transformation is done. Key benefits include:

- **Isolation:** The complex transformations have no effect on performance of the source system.
- **Data Profiling:** Staged data are examined by null ratio check, column length distribution and pattern checks. Some duplication is avoided by truncating staging tables before each load with OnPreExecute event handlers.

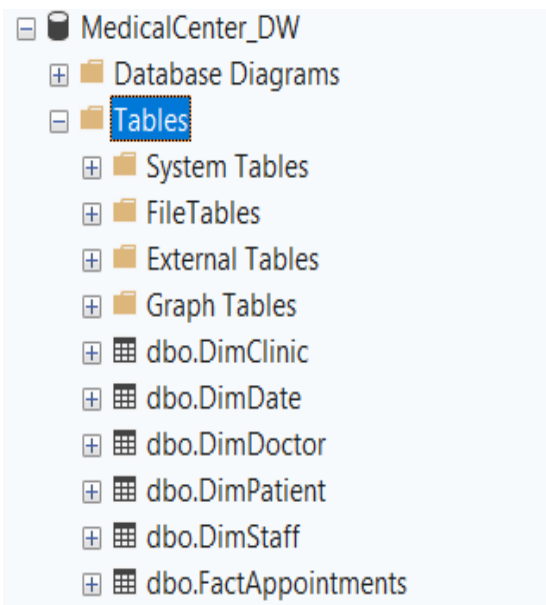
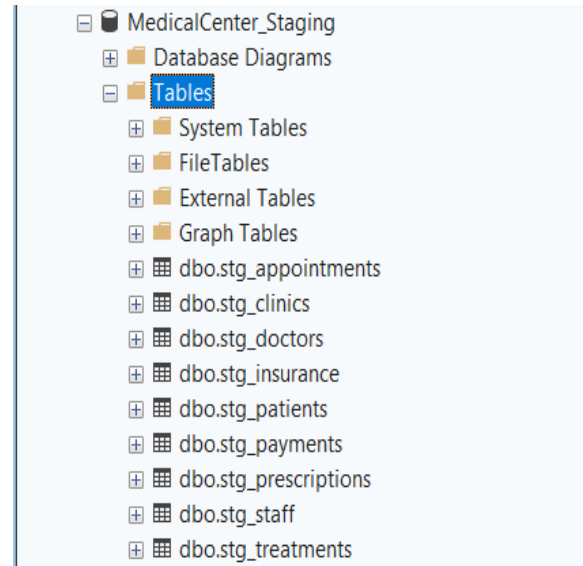
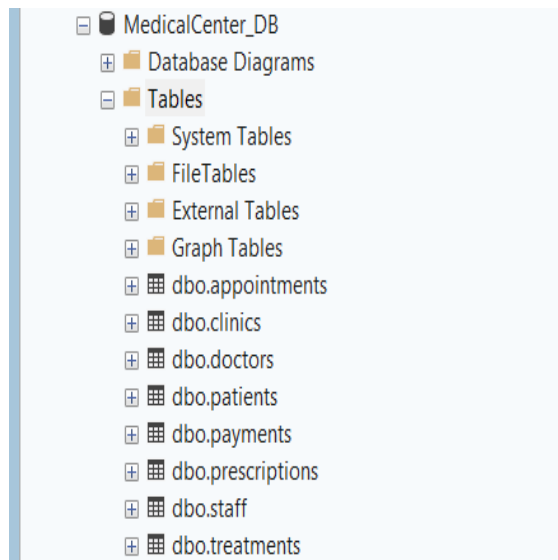
Layer 3 - Data Warehouse (Medicalcenter DW)

The data warehouse is conducted in SQL server star schema. It has one fact table (**FactAppointments**) and five dimension tables (**DimDate**, **DimPatient**, **DimDoctor**, **DimClinic**, **DimStaff**). The stored procedures are used to do the upsert logic and SCD management to load data into the staging.

Layer 4 – BI / Analytics Layer

The last layer is the analytics and reporting layer, which is done using SSAS OLAP cubes, SSRS report, and self-service BI. Aggregated insights are available to end users (business analysts, clinic managers) on appointment trends, revenue analysis, doctor performance, and patient demographics..

4.1 Data Storage Snapshots from SSMS



5 Data Warehouse design and Development

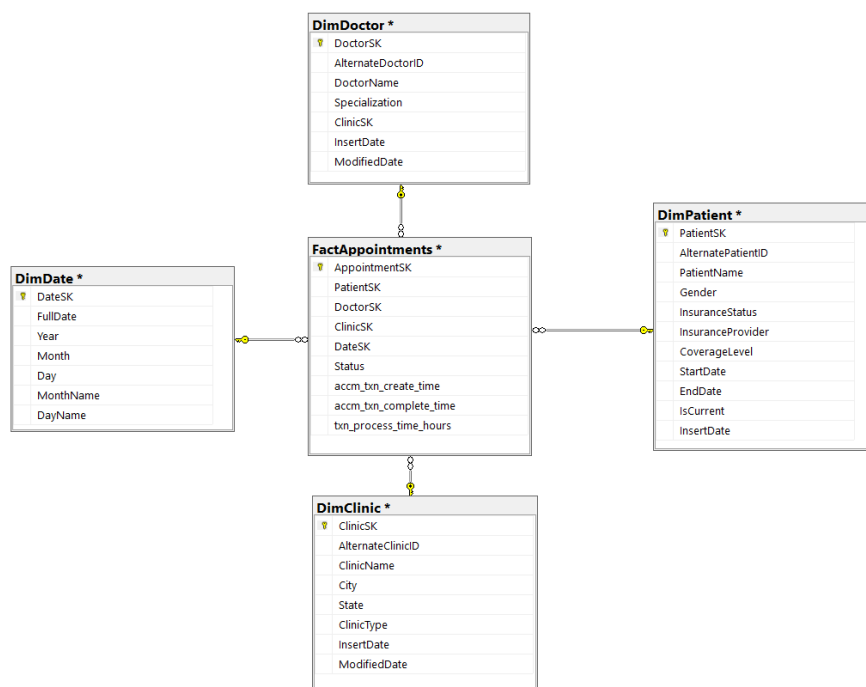
The data warehouse for the Medical Center was designed using a Star Schema to optimize query performance and support multidimensional analysis. The schema consists of one accumulating snapshot fact table and five dimension tables, with DimPatient implemented as a Slowly Changing Dimension (SCD) Type 2.

5.1 Dimensional Model

A **Star Schema** was chosen over a Snowflake Schema for the following reasons:

- Simpler joins and faster query execution for analytical workloads.
- Easier to understand and maintain for business users.
- Optimal for SSAS cube creation and SSRS reporting.
- The dimensional attributes are not overly normalized, making star schema a natural fit.

5.2 Star Schema Diagram



5.3 Fact Table: FactAppointments

The FactAppointments table is designed as an Accumulating Snapshot Fact Table. This fact type is ideal for tracking appointments that progress through defined stages: appointment created → treatment administered → appointment completed

Column	Data Type	Description
AppointmentSK (PK)	INT IDENTITY	Surrogate primary key
PatientSK (FK)	INT	Foreign key to DimPatient
DoctorSK (FK)	INT	Foreign key to DimDoctor
ClinicSK (FK)	INT	Foreign key to DimClinic
DateSK (FK)	INT	Foreign key to DimDate
Status	NVARCHAR(50)	Appointment status (Scheduled/Completed/Cancelled)
PaymentAmount	DECIMAL(10,2)	Payment amount
PaymentMethod	NVARCHAR(50)	Payment method (Cash/Card/Insurance)
PaymentStatus	NVARCHAR(50)	Payment status
TreatmentCost	DECIMAL(10,2)	Treatment cost from stg_treatments
TreatmentType	NVARCHAR(50)	Type of treatment administered
TreatmentStatus	NVARCHAR(50)	Treatment completion status
accm_txn_create_time	DATETIME	Appointment creation timestamp
accm_txn_complete_time	DATETIME	Appointment completion timestamp
txn_process_time_hours	INT	Computed: duration from creation to completion in hours

5.4 Dimension Tables

DimPatient — Contains patient master data including patient name, gender, and insurance-related attributes such as insurance status, provider, and coverage level. Slowly Changing Dimension (Type 2) is implemented to track historical changes in patient information. When a patient's details change, the existing record is expired by updating the EndDate and IsCurrent flag, and a new record is inserted with updated values and a new StartDate. This enables accurate historical analysis of patient data over time.

DimDoctor — Contains doctor reference data including doctor name and specialization. This dimension provides descriptive information about healthcare professionals involved in appointments.

DimClinic — Contains clinic-related reference data including clinic name, city, state, and clinic type. This dimension allows analysis of appointments based on geographic location and clinic characteristics.

DimDate — Contains one record per calendar date and includes attributes such as full date, day, month, year, and quarter. This dimension supports time-based analysis and enables hierarchical reporting across different time periods.

5.6 Hierarchy Support

The following analytical hierarchies are supported in the star schema:

- Date Hierarchy: Year → Month → Day (from DimDate)
- Clinic Hierarchy: ClinicType → ClinicName → City → State
- Doctor Hierarchy: Specialization → DoctorName

Patient Hierarchy: Gender → InsuranceStatus → CoverageLevel (via DimPatient SCD)

5.7 Design Assumptions

- Each record in the fact table represents one appointment between a patient, doctor, and clinic.
- Surrogate keys are used in dimension tables to uniquely identify records and link them with the fact table.
- Slowly Changing Dimension (Type 2) is used in the DimPatient table to keep track of changes in patient details over time.
- The date dimension is created to support analysis based on day, month, quarter, and year.
- A star schema design is used to make querying easier and faster.
- Some columns such as completion time and processing duration are calculated later, so they may be empty initially.

6 ETL Development

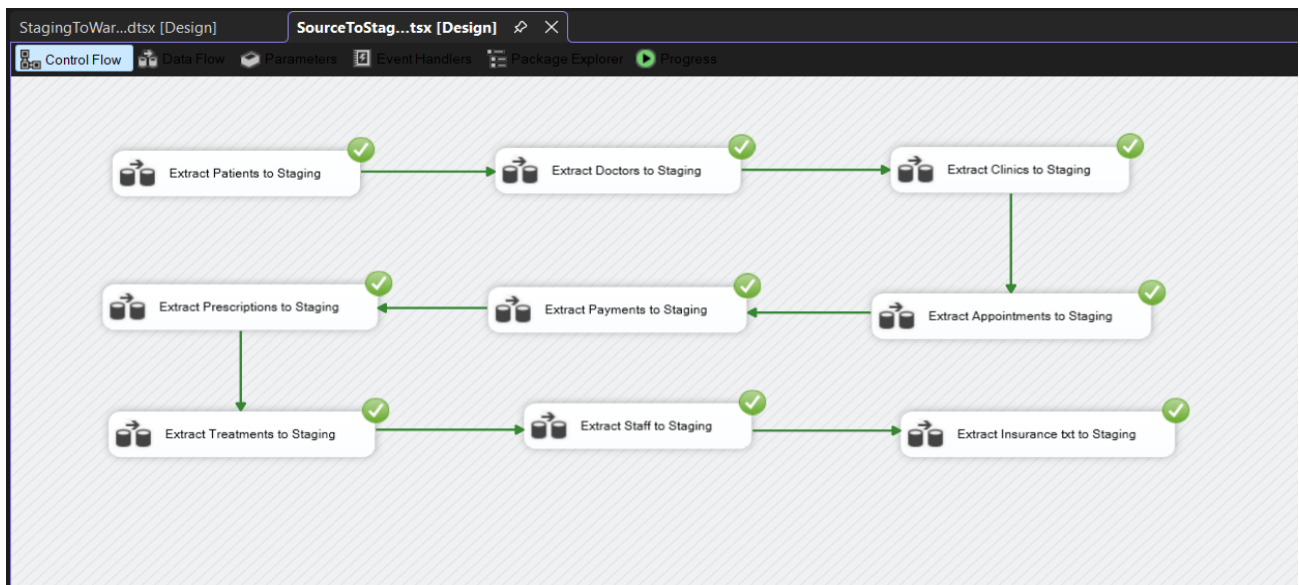
The ETL (Extract, Transform, Load) pipeline was developed using SQL Server Integration Services (SSIS) in SQL Server Data Tools (SSDT). The pipeline consists of two major phases: Source to Staging and Staging to Data Warehouse.

6.1 Data Extraction & Staging Area Loading

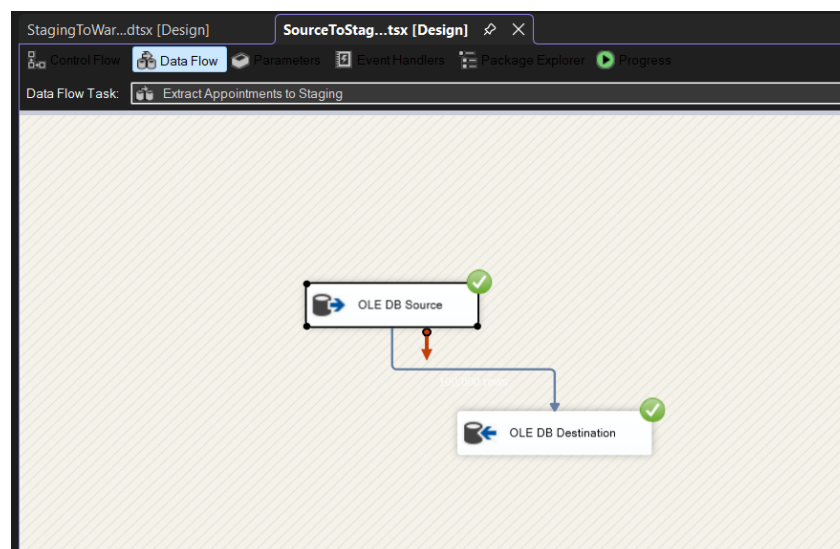
In the first ETL phase, data is extracted from the source OLTP database and the insurance flat file, then loaded into the MedicalCenter_Staging database. Minimal transformations are applied in this phase; the primary goal is data extraction and storage.

6.1.1 Extraction Components Used

Source Type	SSIS Component	Target Staging Table
SQL Server (OLTP)	OLE DB Source	stg_patients, stg_doctors, stg_clinics, stg_appointments, stg_payments, stg_treatments, stg_prescriptions, stg_staff
Flat File (.txt)	Flat File Source	stg_insurance

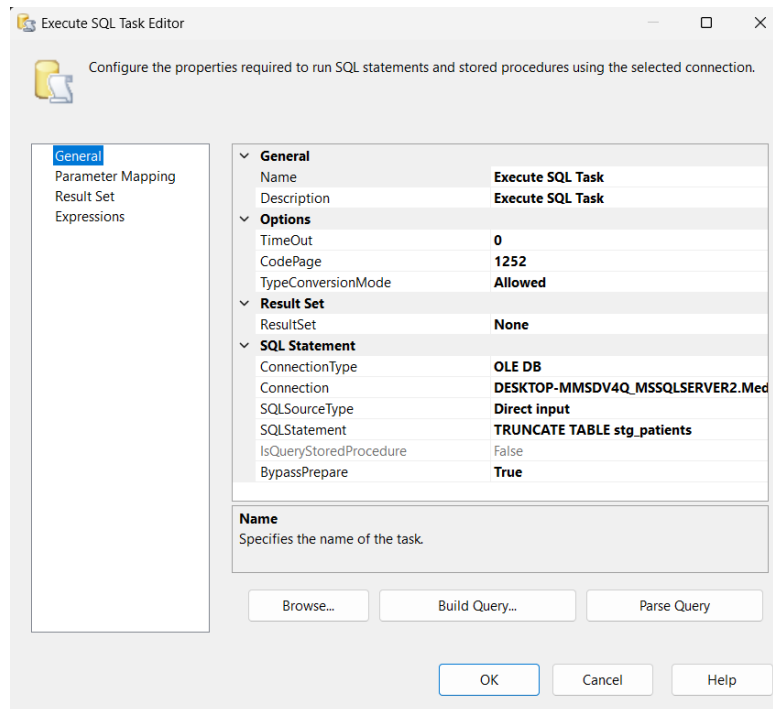


SSIS Control Flow – Source to Staging.dtsx



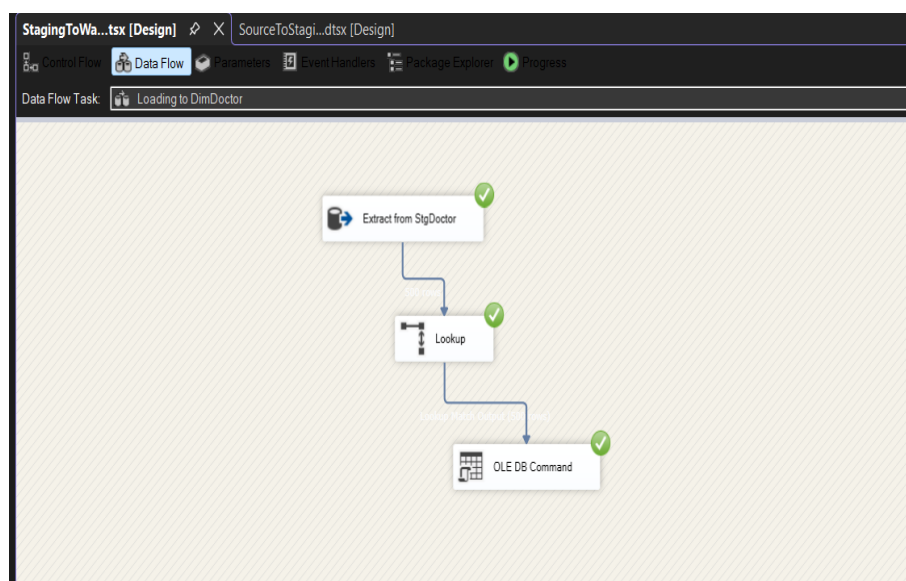
6.1.2 OnPreExecute Event Handler

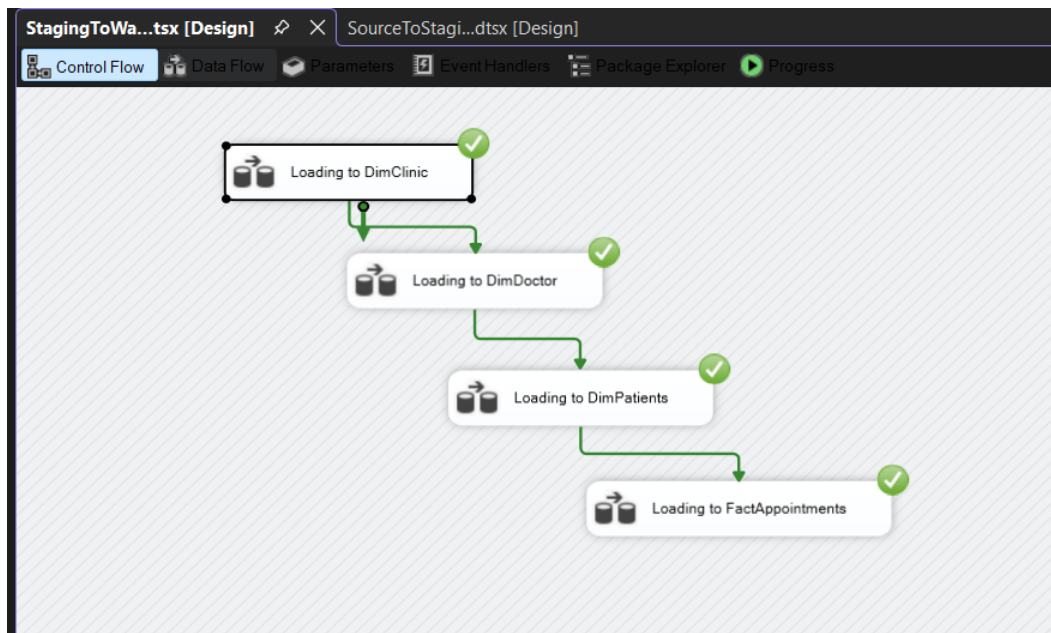
To prevent data duplication when staging tables are repeatedly reloaded, an OnPreExecute Event Handler was added to each Data Flow Task. This handler executes a TRUNCATE TABLE statement on the corresponding staging table before each load begins, ensuring a clean load every time.



6.2 Data Extraction and Data Warehouse Loading

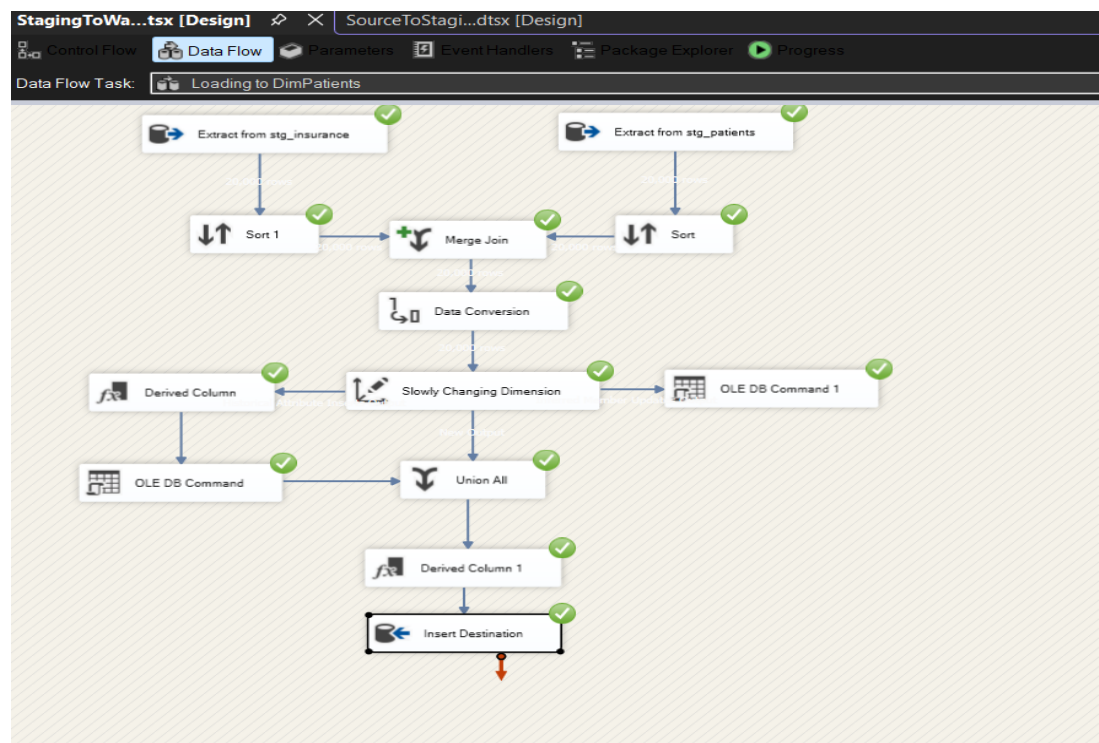
In the second ETL phase, data is extracted from staging tables, transformed, and loaded into the MedicalCenter_DW using stored procedures. The order of execution is critical due to foreign key dependencies between dimension tables and the fact table.





SSIS Control Flow – Staging to Warehouse.dtsx

6.2.1 Slowly Changing Dimension – DimPatient (SCD Type 2)



```

CREATE OR ALTER PROCEDURE dbo.UpdateDimPatient
    @PatientID VARCHAR(10),
    @PatientName NVARCHAR(100),
    @Gender NVARCHAR(10),
    @InsuranceStatus NVARCHAR(50),
    @InsuranceProvider NVARCHAR(50),
    @CoverageLevel NVARCHAR(50)
AS
BEGIN
    IF EXISTS (
        SELECT 1 FROM DimPatient
        WHERE AlternatePatientID = @PatientID AND IsCurrent = 1
    )
    BEGIN
        IF EXISTS (
            SELECT 1 FROM DimPatient
            WHERE AlternatePatientID = @PatientID
            AND IsCurrent = 1
            AND (
                PatientName <> @PatientName OR
                Gender <> @Gender OR
                InsuranceStatus <> @InsuranceStatus OR
                InsuranceProvider <> @InsuranceProvider OR
                CoverageLevel <> @CoverageLevel
            )
        )
        BEGIN
            UPDATE DimPatient
            SET EndDate = GETDATE(),
                IsCurrent = 0
            WHERE AlternatePatientID = @PatientID
            AND IsCurrent = 1;

            INSERT INTO DimPatient
            (
                AlternatePatientID,
                PatientName,
                Gender,
                InsuranceStatus,
                InsuranceProvider,
                CoverageLevel,
                StartDate,
                EndDate,
                IsCurrent
            )
            VALUES
            (
                @PatientID,
                @PatientName,
                @Gender,
                @InsuranceStatus,
                @InsuranceProvider,
                @CoverageLevel,
                GETDATE(),
                NULL,
                1
            );
        END
    END
    ELSE
    BEGIN
        INSERT INTO DimPatient
        (
            AlternatePatientID,
            PatientName,
            Gender,
            InsuranceStatus,
            InsuranceProvider,
            CoverageLevel,
            StartDate,
            EndDate,
            IsCurrent
        )
        VALUES
        (
            @PatientID,
            @PatientName,
            @Gender,
            @InsuranceStatus,
            @InsuranceProvider,
            @CoverageLevel,
            GETDATE(),
            NULL,
            1
        );
    END
END

```

```

-- SP: UpdateDimClinic
=====
CREATE OR ALTER PROCEDURE dbo.UpdateDimClinic
    @ClinicID VARCHAR(10),
    @ClinicName NVARCHAR(100),
    @City NVARCHAR(50),
    @State NVARCHAR(50),
    @ClinicType NVARCHAR(50)
AS
BEGIN
    -- INSERT if NOT EXISTS
    IF NOT EXISTS (
        SELECT 1
        FROM DimClinic
        WHERE AlternateClinicID = @ClinicID
    )
    BEGIN
        INSERT INTO DimClinic
        (
            AlternateClinicID,
            ClinicName,
            City,
            State,
            ClinicType,
            InsertDate
        )
        VALUES
        (
            @ClinicID,
            @ClinicName,
            @City,
            @State,
            @ClinicType,
            GETDATE()
        )
    END
    -- UPDATE if EXISTS
    ELSE
    BEGIN
        UPDATE DimClinic
        SET
            ClinicName = @ClinicName,
            City = @City,
            State = @State,
            ClinicType = @ClinicType,
            ModifiedDate = GETDATE()
        WHERE AlternateClinicID = @ClinicID
    END
END

```

```

-- SP: UpdateDimDoctor
=====
CREATE OR ALTER PROCEDURE dbo.UpdateDimDoctor
    @DoctorID VARCHAR(10),
    @DoctorName NVARCHAR(100),
    @Specialization NVARCHAR(100),
    @ClinicSK INT
AS
BEGIN
    -- If doctor does NOT exist → INSERT
    IF NOT EXISTS (
        SELECT 1
        FROM DimDoctor
        WHERE AlternateDoctorID = @DoctorID
    )
    BEGIN
        INSERT INTO DimDoctor
        (
            AlternateDoctorID,
            DoctorName,
            Specialization,
            ClinicSK,
            InsertDate
        )
        VALUES
        (
            @DoctorID,
            @DoctorName,
            @Specialization,
            @ClinicSK,
            GETDATE()
        )
    END
    -- If doctor exists → UPDATE
    ELSE
    BEGIN
        UPDATE DimDoctor
        SET
            DoctorName = @DoctorName,
            Specialization = @Specialization,
            ClinicSK = @ClinicSK,
            ModifiedDate = GETDATE()
        WHERE AlternateDoctorID = @DoctorID
    END
END

```

```

CREATE PROCEDURE dbo.LoadFactAppointment
    @AppointmentID VARCHAR(10),
    @AppointmentDate DATE,
    @PatientID VARCHAR(10),
    @DoctorID VARCHAR(10),
    @ClinicID VARCHAR(10),
    @Status NVARCHAR(50),
    @ReasonForVisit NVARCHAR(100),
    @PaymentAmount DECIMAL(10,2),
    @PaymentMethod NVARCHAR(50),
    @PaymentStatus NVARCHAR(50),
    @TreatmentCost DECIMAL(10,2),
    @TreatmentType NVARCHAR(50),
    @TreatmentStatus NVARCHAR(50)
AS
BEGIN
    -- Resolve surrogate keys
    DECLARE @DateSK INT = CONVERT(INT, CONVERT(CHAR(8), @AppointmentDate, 112));
    DECLARE @PatientSK INT = NULL;
    DECLARE @DoctorSK INT = NULL;
    DECLARE @ClinicSK INT = NULL;

    SELECT @PatientSK = PatientSK FROM DimPatient
    WHERE AlternatePatientID = @PatientID AND IsCurrent = 1;

    SELECT @DoctorSK = DoctorSK FROM DimDoctor
    WHERE AlternateDoctorID = @DoctorID;

    SELECT @ClinicSK = ClinicSK FROM DimClinic
    WHERE AlternateClinicID = @ClinicID;

    -- Only insert if not already loaded
    IF NOT EXISTS (SELECT 1 FROM FactAppointment WHERE AppointmentID = @AppointmentID)
    BEGIN
        INSERT INTO FactAppointment
        (
            AppointmentID, DateSK, PatientSK, DoctorSK, ClinicSK,
            Status, ReasonForVisit,
            PaymentAmount, PaymentMethod, PaymentStatus,
            TreatmentCost, TreatmentType, TreatmentStatus,
            accn_txn_create_time, InsertDate, ModifiedDate
        )
        VALUES (
            @AppointmentID, @DateSK, @PatientSK, @DoctorSK, @ClinicSK,
            @Status, @ReasonForVisit,
            ISNULL(@PaymentAmount, 0), @PaymentMethod, @PaymentStatus,
            ISNULL(@TreatmentCost, 0), @TreatmentType, @TreatmentStatus,
            GETDATE(), GETDATE(), GETDATE()
        );
    END
END
GO

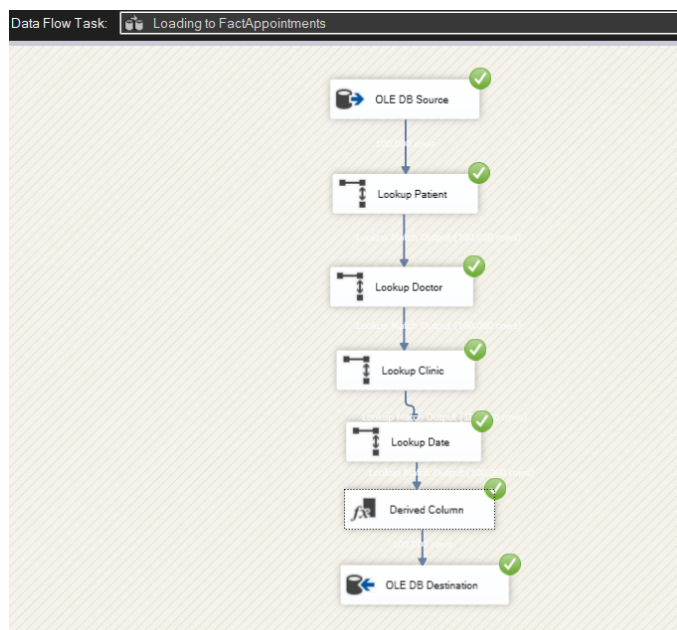
```

6.2.2 Fact Table Loading – LoadFactAppointment

The FactAppointments table is loaded by joining data from stg_appointments, stg_payments, and stg_treatments in the staging database. The LoadFactAppointment stored procedure resolves all surrogate keys before inserting.

Surrogate Key Resolution Process:

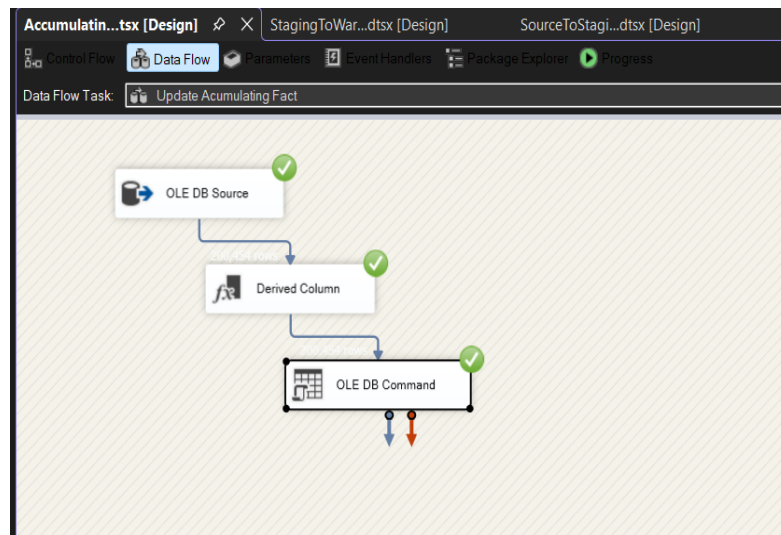
- DateSK: Computed as CONVERT(INT, CONVERT(CHAR(8), @AppointmentDate, 112)) – formats the date as YYYYMMDD integer.
- PatientSK: Looked up from DimPatient WHERE AlternatePatientID = @PatientID AND IsCurrent = 1.
- DoctorSK: Looked up from DimDoctor WHERE AlternateDoctorID = @DoctorID.
- ClinicSK: Looked up from DimClinic WHERE AlternateClinicID = @ClinicID.



6.2.3 UpdateAccumulatingFact

The FactAppointments table implements an Accumulating Snapshot pattern to track the lifecycle of each appointment.

- accm_txn_create_time: Set at the time of initial fact row insertion (GETDATE() in LoadFactAppointment).
- accm_txn_complete_time: Updated via UpdateAccumulatingFact when the appointment is marked as complete.
- txn_process_time_hours: A computed column (or calculated field) that derives the total processing time in hours from creation to completion.



Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

- Variables and Parameters
- Columns

- Mathematical Functions
- String Functions
- Date/Time Functions
- NULL Functions
- Type Casts
- Operators

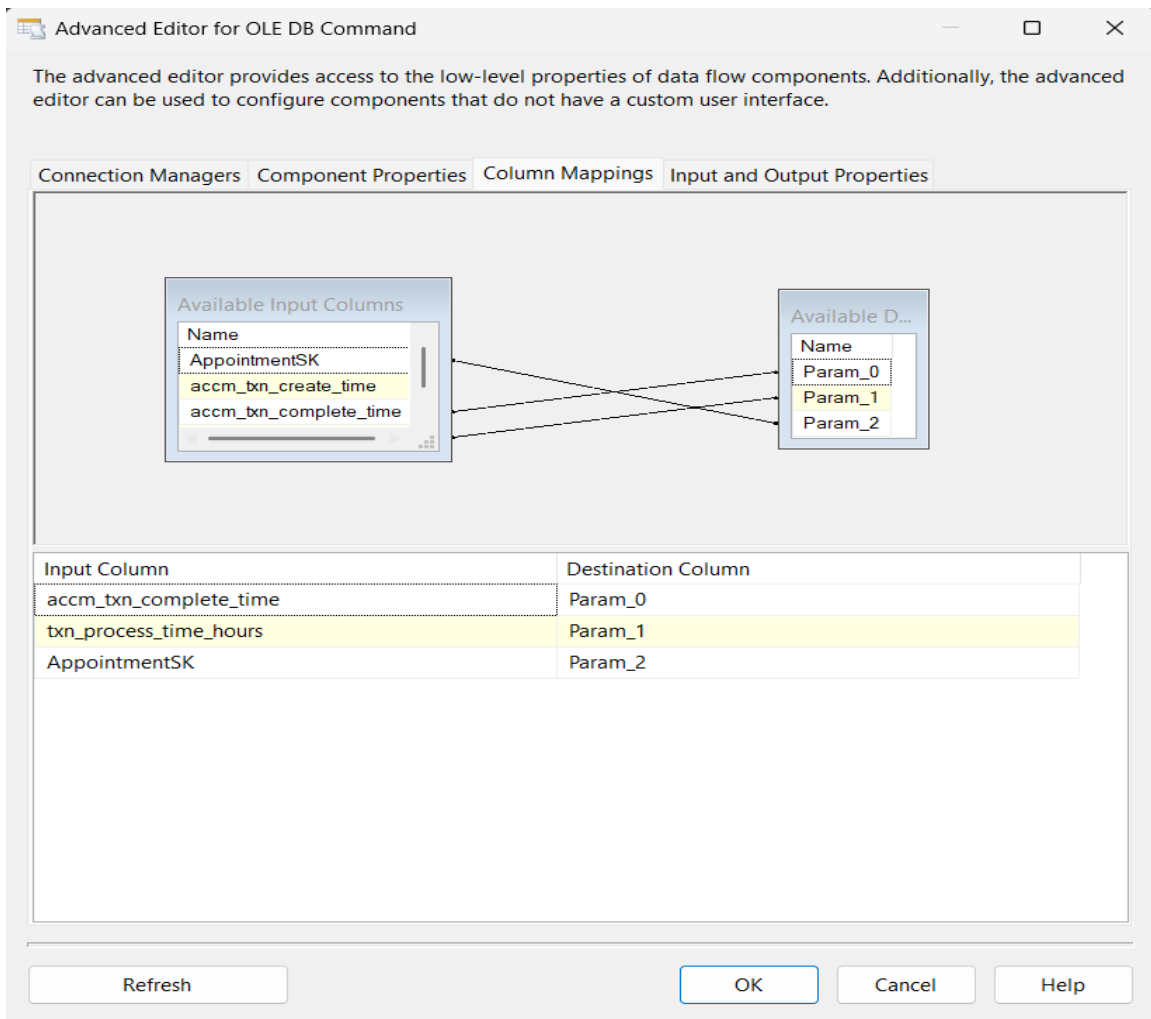
Description:

Derived Column Name	Derived Column	Expression	Data Type
accm_txn_complete_t...	<add as new column>	GETDATE()	database timestamp ...
txn_process_time_ho...	<add as new column>	DATEDIFF("hour",accm_txn_create_time,GETD...	four-byte signed inte...

Configure Error Output... OK Cancel Help

The OLE DB Command component executes the following parameterized UPDATE for each matched row:

```
UPDATE FactAppointments
SET
    accm_txn_complete_time = ?,
    txn_process_time_hours = ?
WHERE AppointmentSK = ?
```

7 Conclusion

- A suitable OLTP dataset was selected based on a Healthcare Clinic Management System scenario, covering patients, doctors, clinics, appointments, payments, treatments, prescriptions, and staff.
- A supplementary insurance flat file was incorporated as a secondary heterogeneous data source alongside the main SQL Server database.
- Data was successfully extracted from both the source database and the flat file and loaded into the staging area (MedicalCenter_Staging) using SSIS.
- Data profiling was performed on the staged data to identify null values, column distributions, and data quality issues before loading into the data warehouse.
- The data warehouse (MedicalCenter_DW) was designed using a Star Schema consisting of one fact table and five dimension tables.
- DimPatient was implemented as a Slowly Changing Dimension Type 2 to maintain historical records of patient insurance information.
- FactAppointments was designed as an Accumulating Snapshot Fact Table to track the full lifecycle of each appointment from creation to completion.
- Stored procedures were developed for each dimension and fact table to handle upsert logic and surrogate key resolution during the ETL process.
- The complete ETL pipeline was orchestrated using SSIS with proper execution order and event handlers to prevent data duplication.
- The implemented solution provides a strong foundation for future SSAS cube development, SSRS reporting, and business intelligence analysis within the healthcare clinic domain.